

Solution Architecture - HouseRent MERN

Date: 18 June 2026

Project Name: House Rent

Maximum Marks: 4 Marks

SOLUTION ARCHITECTURE OVERVIEW

Architecture Type: 3-Tier Client-Server

[PRESENTATION TIER]

React Frontend (Vite, Bootstrap, Material UI)

Runs on: `http://localhost:5174`

|

| REST API (HTTP)

|

[APPLICATION TIER]

Express Backend (Node.js)

Runs on: `http://localhost:8000`

|

| MongoDB Queries

|

[DATA TIER]

MongoDB Database (Cloud Atlas)

FRONTEND LAYER:

- React 18 components for dynamic UI
- Vite for fast build and HMR
- Bootstrap 5 for responsive design
- Material UI for advanced components
- Axios for API communication
- React Router for navigation

Components:

- Home (landing page)
- Login/Register (authentication)
- RenterHome (renter dashboard)
- AllProperties (property listing)
- PropertyDetail (detailed view)
- OwnerHome (owner dashboard)
- AdminHome (admin dashboard)

BACKEND LAYER:

- Node.js runtime environment
- Express.js web framework
- JWT authentication
- bcryptjs password hashing

Controllers:

- authController (registration, login)
- userController (user management)
- propertyController (CRUD, search, filter)
- bookingController (bookings, approvals)
- adminController (approvals, oversight)

Routes:

- `/api/auth` (login, register)
- `/api/properties` (search, filter, CRUD)
- `/api/bookings` (create, manage, approve)
- `/api/admin` (user approval, statistics)

DATABASE LAYER:

- MongoDB (NoSQL document database)
- Mongoose for ODM and schema validation

- MongoDB Atlas for cloud hosting

Collections:

- Users (profiles, roles, credentials)
- Properties (listings, owner reference)
- Bookings (requests, statuses)
- BookingHistory (search history)

DATA FLOW:

1. Renter searches properties via React UI
2. Axios sends GET /api/properties with filters
3. Express validates JWT and processes request
4. Mongoose queries MongoDB for matching properties
5. Results returned to frontend as JSON
6. React renders property list

SECURITY:

JWT tokens for stateless authentication
bcrypt password hashing (never plain text)
CORS configured for frontend access
Role-based middleware for access control
Input validation on all endpoints

DEPLOYMENT:

Dev: npm run dev (frontend & backend locally)
Prod: Build frontend, deploy to CDN/hosting
Deploy backend to Node.js hosting
Use MongoDB Atlas for database